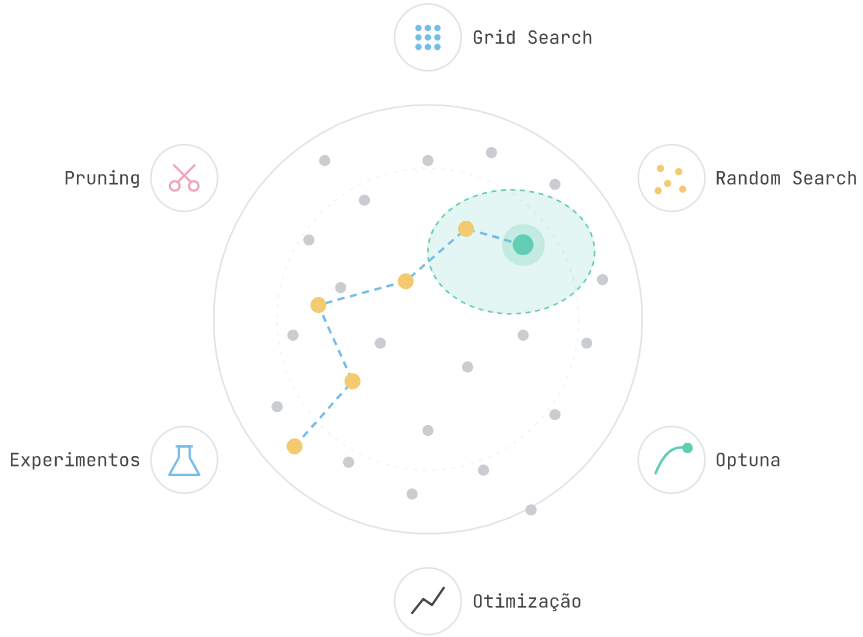
AULA 7.5 · MÓDULO 7

OTIMIZAÇÃO DE HIPERPARÂMETROS

Como encontrar boas configurações para modelos de Machine Learning.

Grid Search Random Search Optuna



VISÃO GERAL · 120 MIN

Roteiro da aula

8 min	Revisão e introdução	10 min	Parâmetros, hiperparâmetros e espaço de busca	12 min	Grid Search
12 min	Random Search	18 min	Otimização adaptativa, TPE e Optuna	12 min	Pruning e MedianPruner
30 min	**Exercício guiado: mesmo orçamento, três estratégias**	5 min	Quiz	3 min	AutoML e encerramento

BLOCO 1 · RECAPITULAÇÃO

O Módulo 7 até aqui

7.1

Regularização

`alpha` `l1_ratio`

Ridge, Lasso e ElasticNet encolhem coeficientes; alpha define a intensidade.

7.2

SVM

`C` `gamma`

C pune erros de classificação; gamma controla o alcance do kernel e o recorte da fronteira.

7.3

Random Forest

`n_estimators` `max_depth` `max_features`

Quantas árvores, quão profundas e quão diferentes entre si.

7.4

Boosting

`learning_rate` `max_depth` `n_estimators`

Passos menores costumam exigir mais árvores; os três interagem.

Em todas as aulas, o desempenho dependeu de valores escolhidos **antes** do `fit()`.

A PERGUNTA DA AULA

Como escolher boas configurações sem testar valores ao acaso?

espaço de busca + métrica de validação + orçamento + estratégia =

otimização de hiperparâmetros

BLOCO 2 · FUNDAMENTOS

Parâmetros e hiperparâmetros

Hiperparâmetros

Definidos por nós, antes do ajuste

`max_depth` `n_estimators` `learning_rate` `min_samples_leaf` `C` `gamma`

`fit(X, y)`

minimiza a perda no treino

Parâmetros

Aprendidos pelo modelo, durante o ajuste

Coeficientes de uma regressão

Pesos de uma rede neural

Divisões e valores das folhas

Se o treino escolhesse `max_depth`, escolheria sempre a maior profundidade: ela reduz o erro de treino. Escolher hiperparâmetros exige um critério externo, calculado em dados de validação.

BLOCO 2 · FUNDAMENTOS

Escolher, treinar, avaliar, otimizar

`laço externo · otimizar hiperparâmetros (repete para cada θ)`

1 · Escolher
uma configuração θ
`max_depth=3`

→

`laço interno`

2 · Treinar
fit() ajusta os parâmetros w com θ fixo, minimizando a perda de treino

→

3 · Avaliar
na validação, gerando J(θ)
`AUC = 0,91`

4 · Otimizar comparar os valores de J(θ), propor a próxima configuração e, ao final, ficar com a melhor.

Valores ilustrativos. Cada volta do laço externo custa pelo menos um treinamento completo; com validação cruzada de k folds, custa k.

BLOCO 3 · FORMULAÇÃO

A otimização como um problema de argmax

$$\theta^* = \arg\max_{\theta \in \Theta} J(\theta)$$

θ	Θ	J	$\arg\max$	θ^*
uma configuração de hiperparâmetros	espaço de busca: as configurações permitidas	função objetivo: métrica de validação obtida ao treinar com θ	devolve a configuração, não o valor máximo	configuração selecionada

J não tem fórmula fechada nem gradiente: cada avaliação exige treinar modelos. Por isso usamos estratégias de busca.

BLOCO 3 · FORMULAÇÃO

Maximizar ou minimizar

$$\theta^* = \arg\max J(\theta)$$

Métricas de acerto: quanto maior, melhor

ROC AUC | F1 | Acurácia

$$\theta^* = \arg\min L(\theta)$$

Métricas de erro: quanto menor, melhor

Log Loss | Erro quadrático médio

$$\arg\min L = \arg\max (-L)$$

O scikit-learn sempre maximiza o score; por isso as perdas aparecem como `neg_log_loss`. No Optuna, informamos `direction`.

BLOCO 4 · ESPAÇO DE BUSCA

O espaço de busca define o que pode ser testado

```
param_grid = {
    "max_depth": [3, 5, 7],
    "min_samples_leaf": [1, 2],
    "n_estimators": [100, 200]
}
```

$$|\Theta| = 3 \times 2 \times 2 = 12$$

Cada configuração escolhe um valor de cada lista, de forma independente: princípio multiplicativo.

	leaf=1 n=100	leaf=1 n=200	leaf=2 n=100	leaf=2 n=200
depth=3	θ_1	θ_2	θ_3	θ_4
depth=5	θ_5	θ_6	θ_7	θ_8
depth=7	θ_9	θ_{10}	θ_{11}	θ_{12}

BLOCO 4 · ESPAÇO DE BUSCA

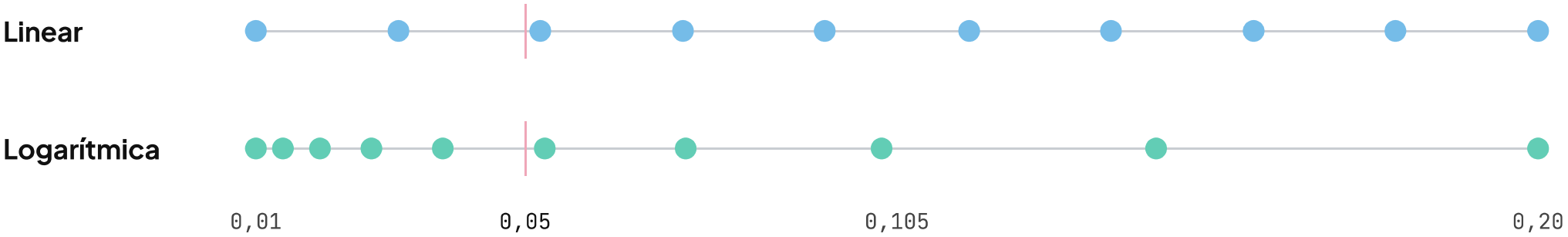
Tipos de hiperparâmetros e como descrevê-los

TIPO	EXEMPLO	SCIKIT-LEARN / SCIPY	OPTUNA · BLOCO 18
Categórico	`max_features`	`["sqrt", None]`	`suggest_categorical`
Inteiro	`max_depth ∈ 1…15`	`randint(1, 16)`	`suggest_int(…, 1, 15)`
Contínuo linear	`subsample ∈ [0,5; 1]`	`uniform(0.5, 0.5)`	`suggest_float`
Contínuo log	`learning_rate ∈ [0,01; 0,2]`	`loguniform(0.01, 0.2)`	`suggest_float(…, log=True)`

Atenção: `randint(a, b)` exclui b; `uniform(loc, scale)` cobre [loc, loc + scale]; `suggest_int` inclui os dois extremos.

BLOCO 4 · ESPAÇO DE BUSCA

Escala linear e escala logarítmica



Uniforme: P(η < 0,05)

$$(0{,}05 - 0{,}01) / 0{,}19 \approx \mathbf{21\%}$$

Log-uniforme: P(η < 0,05)

$$\ln(5) / \ln(20) \approx 1{,}609 / 2{,}996 \approx \mathbf{54\%}$$

BLOCO 5 · GRID SEARCH

Grid Search: busca exaustiva numa grade

01 Definir os hiperparâmetros

02 Definir os valores possíveis de cada um

03 Gerar todas as combinações

04 Treinar um modelo por combinação e fold

05 Avaliar cada configuração pela média dos folds

06 **Selecionar a de maior média**



A lista inteira é decidida antes de começar. Resultados anteriores não mudam o que será testado.

BLOCO 6 · GRID SEARCH

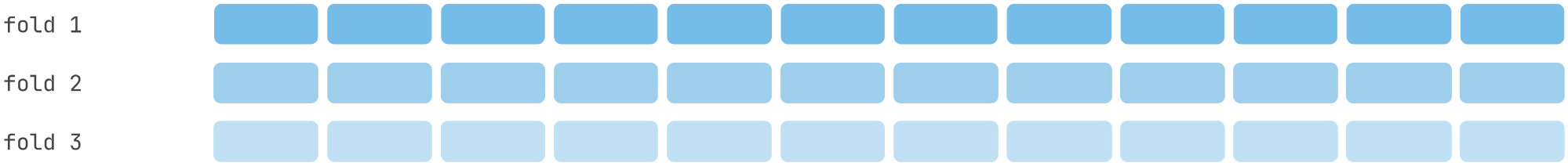
Configurações, folds e ajustes de modelo

```
param_grid = {
    "max_depth": [3, 5, 7],
    "min_samples_leaf": [1, 2],
    "max_features": ["sqrt", None]
}
```

$N = 3 \times 2 \times 2 = 12$ configurações

12×3 folds = **36** ajustes na busca

+ 1 ajuste final do melhor modelo se `refit=True`



12 configurações × 3 folds: cada quadrado é um `fit()`

BLOCO 7 · GRID SEARCH

GridSearchCV no scikit-learn

```python
from sklearn.model_selection import GridSearchCV
from sklearn.ensemble import RandomForestClassifier

modelo = RandomForestClassifier(
    n_estimators=200, random_state=42
)

grid = GridSearchCV(
    estimator=modelo,
    param_grid=param_grid,
    scoring="roc_auc",
    cv=3,
    n_jobs=-1,
    refit=True,
    verbose=1
)
grid.fit(X_train, y_train)
```

`estimator`	modelo base com o que não varia
`param_grid`	valores de cada hiperparâmetro
`scoring`	métrica a maximizar na validação
`cv`	número de folds, objeto ou lista de divisões
`n_jobs`	processos em paralelo; -1 usa todos
`refit`	retreina o melhor com todos os dados do fit
`verbose`	quantidade de mensagens impressas

BLOCO 7 · GRID SEARCH

Lendo os resultados da busca

```
grid.best_params_   # configuração vencedora
grid.best_score_    # média na validação

resultados = pd.DataFrame(grid.cv_results_)
resultados[["params", "mean_test_score",
            "std_test_score", "rank_test_score"]]
```

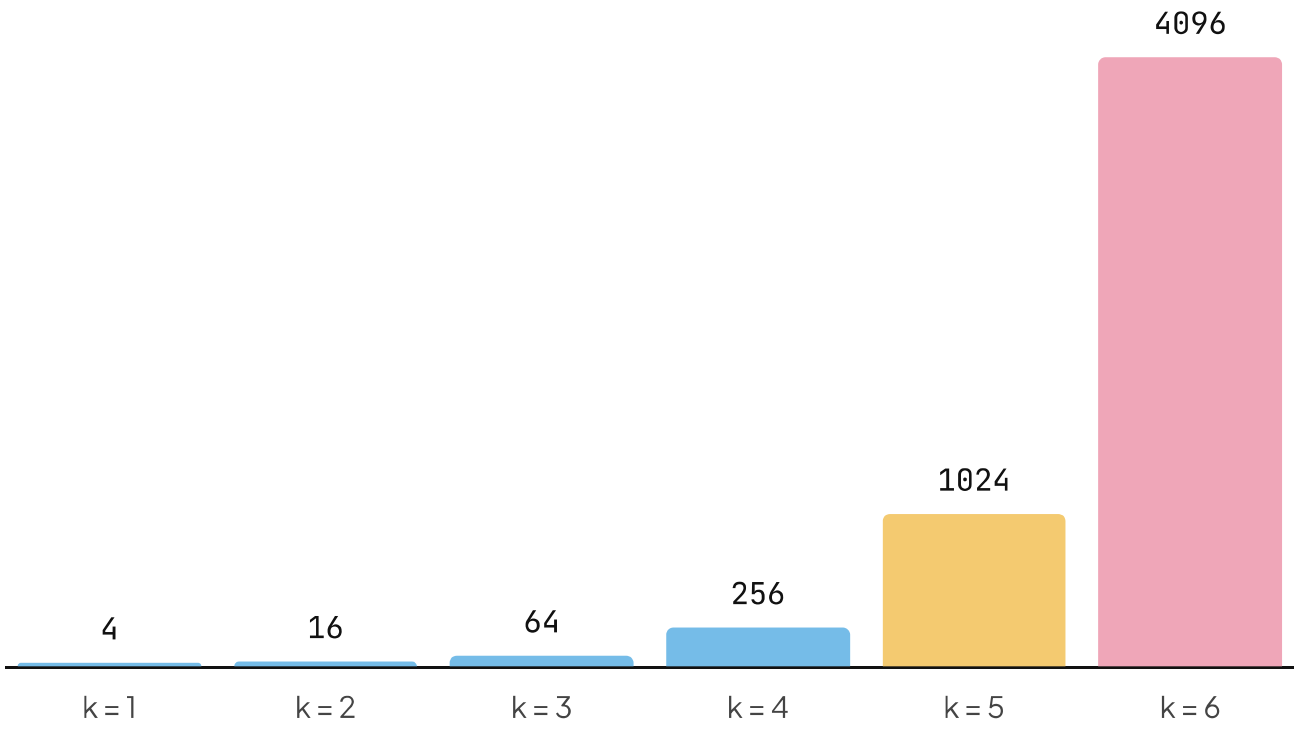
No `cv_results_`, *test_score* é o fold de validação de cada divisão, não o conjunto de teste final.

COLUNAS DE cv_results_

`params`	configuração
`mean_test_score`	média dos folds
`std_test_score`	desvio entre folds
`rank_test_score`	posição
`split0_test_score`	fold 1
`mean_fit_time`	tempo médio de ajuste

BLOCO 8 · GRID SEARCH

O tamanho da grade cresce exponencialmente



$$N = 4^k$$

k hiperparâmetros, 4 valores cada.
Altura das barras proporcional ao valor.

Com k = 6 e 5 folds:

$$4096 \times 5 = \mathbf{20\,480}$$

ajustes

BLOCO 8 · GRID SEARCH

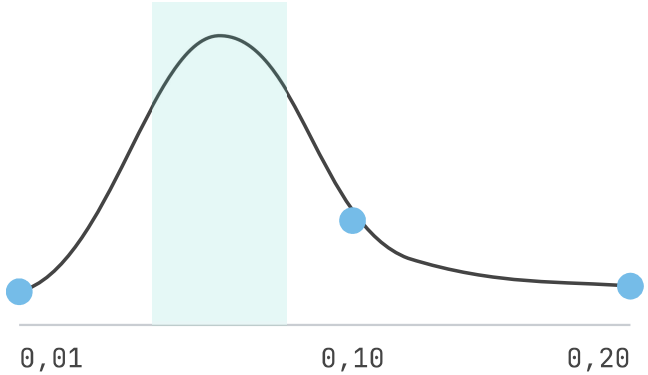
Vantagens e limitações do Grid Search

VANTAGENS

- Sistemático
- Fácil de compreender
- Avalia toda a grade
- Reproduzível com protocolo controlado
- Paraleliza bem

LIMITAÇÕES

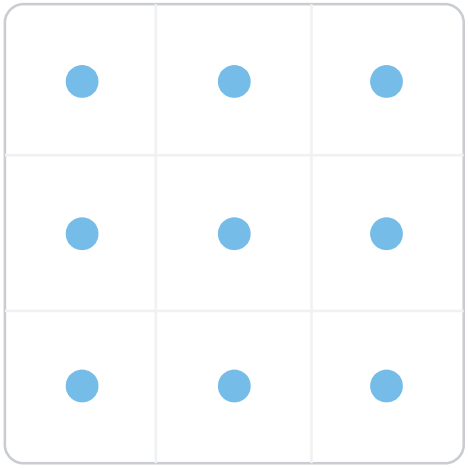
- Custo elevado em espaços grandes
- Depende dos valores escolhidos
- Ignora regiões intermediárias
- Crescimento combinatório



Curva hipotética. Os três pontos da grade não alcançam a região verde.

BLOCO 9 · RANDOM SEARCH

Random Search: sortear configurações



Grade fixa · 9 avaliações

Aleatória · 9 avaliações

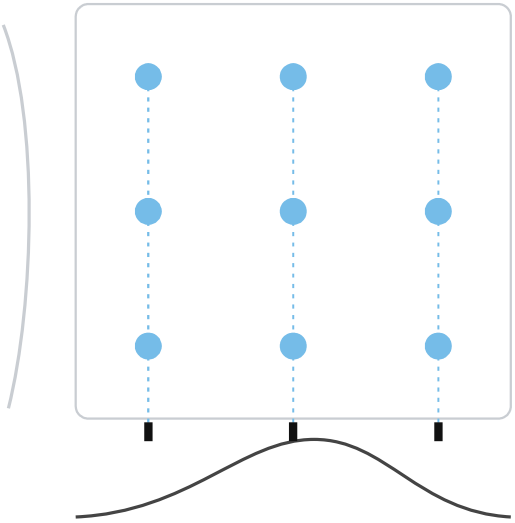
Amostra `n_iter` configurações a partir de distribuições definidas por nós.

O custo depende de `n_iter`, não do tamanho do espaço.

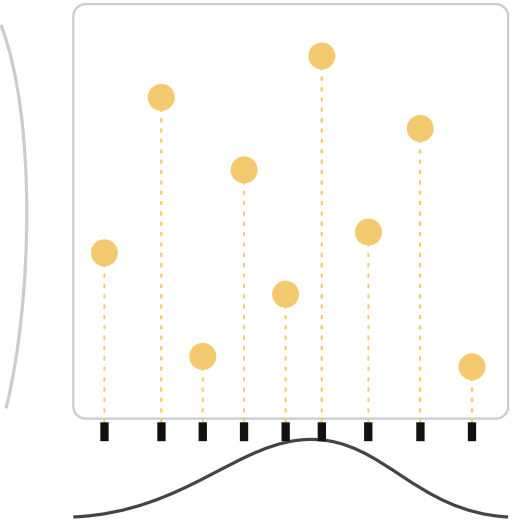
Não percorre obrigatoriamente todas as regiões.

BLOCO 10 · RANDOM SEARCH

Mais valores distintos nas dimensões que importam



Grade: 3 valores distintos



Aleatória: 9 valores distintos

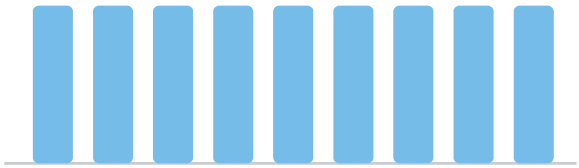
Eixo horizontal: hiperparâmetro importante. A curva abaixo varia muito.

Eixo vertical: pouco importante. A curva lateral é quase plana.

Vantagem estatística, não garantia: o sorteio pode deixar de lado a melhor região.

BLOCO 11 · RANDOM SEARCH

Distribuições descrevem o espaço de busca



Discreta uniforme

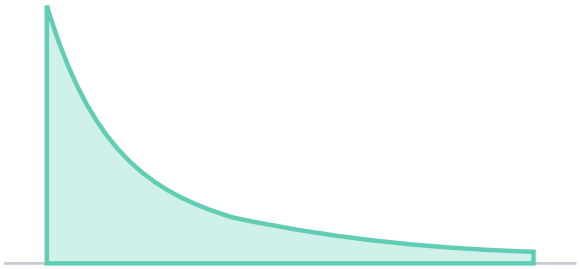
`randint(1, 10)`

Inteiros de 1 a 9, cada um com probabilidade 1/9

Uniforme contínua

`uniform(0.5, 0.5)`

Qualquer valor em [0,5; 1,0], densidade constante



Log-uniforme

`loguniform(0.01, 0.2)`

log(x) é uniforme; no eixo linear, densidade ∝ 1/x

BLOCO 11 · RANDOM SEARCH

Distribuições no código

```python
from scipy.stats import randint, loguniform

# espaço para um modelo de boosting
param_distributions = {
    "max_depth": randint(3, 15),
    "learning_rate": loguniform(0.01, 0.2),
    "min_samples_leaf": randint(1, 10)
}

# cada distribuição sabe sortear valores
loguniform(0.01, 0.2).rvs(5, random_state=0)
```

`randint(3, 15)`
inteiros de 3 a 14; o limite superior é exclusivo

`loguniform(0.01, 0.2)`
contínuo; mesma atenção a cada ordem de grandeza

`randint(1, 10)`
inteiros de 1 a 9

`learning_rate` existe em modelos de boosting; uma Random Forest não o possui.

BLOCO 12 · RANDOM SEARCH

RandomizedSearchCV no scikit-learn

```
from sklearn.model_selection import (
    RandomizedSearchCV
)

busca = RandomizedSearchCV(
    estimator=modelo,
    param_distributions=param_distributions,
    n_iter=12,
    scoring="roc_auc",
    cv=3,
    random_state=42,
    n_jobs=1,
    refit=False
)
busca.fit(X_train, y_train)
```

`n_iter`

quantas configurações sortear: o orçamento

`random_state`

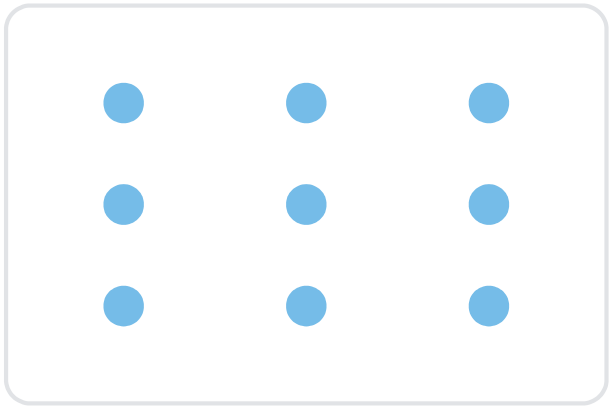
mesma semente, mesmas configurações

12 × 3 = 36 ajustes

Com distribuições contínuas, os valores são sorteados, não tirados de uma grade fixa.

BLOCO 13 · BUSCA ADAPTATIVA

Aprender com os experimentos anteriores

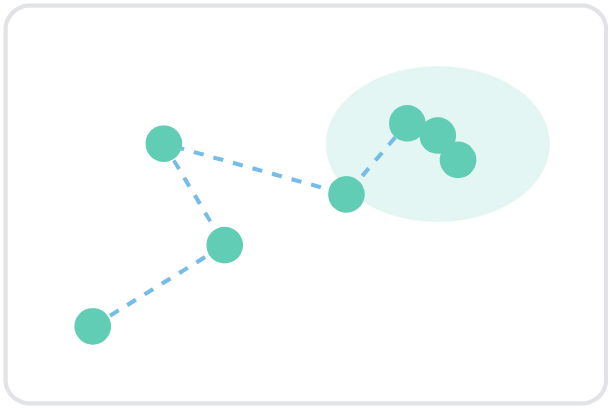


Exaustiva

Lista fixa, definida antes de começar

Aleatória

Cada sorteio ignora os anteriores



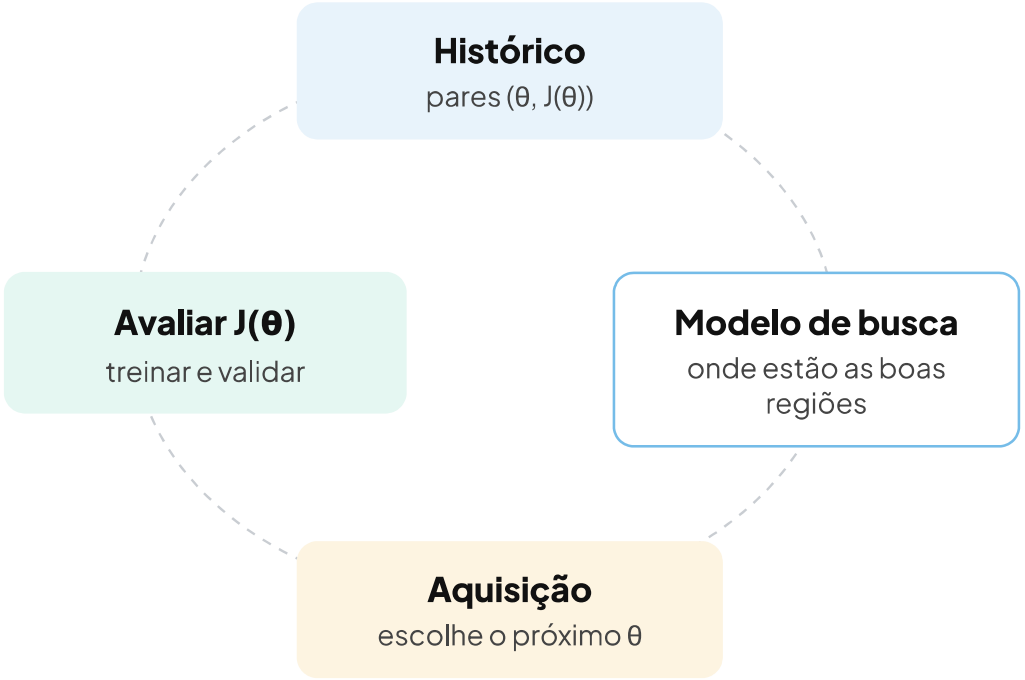
Sequencial adaptativa

O próximo candidato depende do histórico

Configuração → Resultado → Aprendizado sobre o espaço → Nova configuração ↺

BLOCO 14 · OTIMIZAÇÃO BAYESIANA

O ciclo da otimização bayesiana



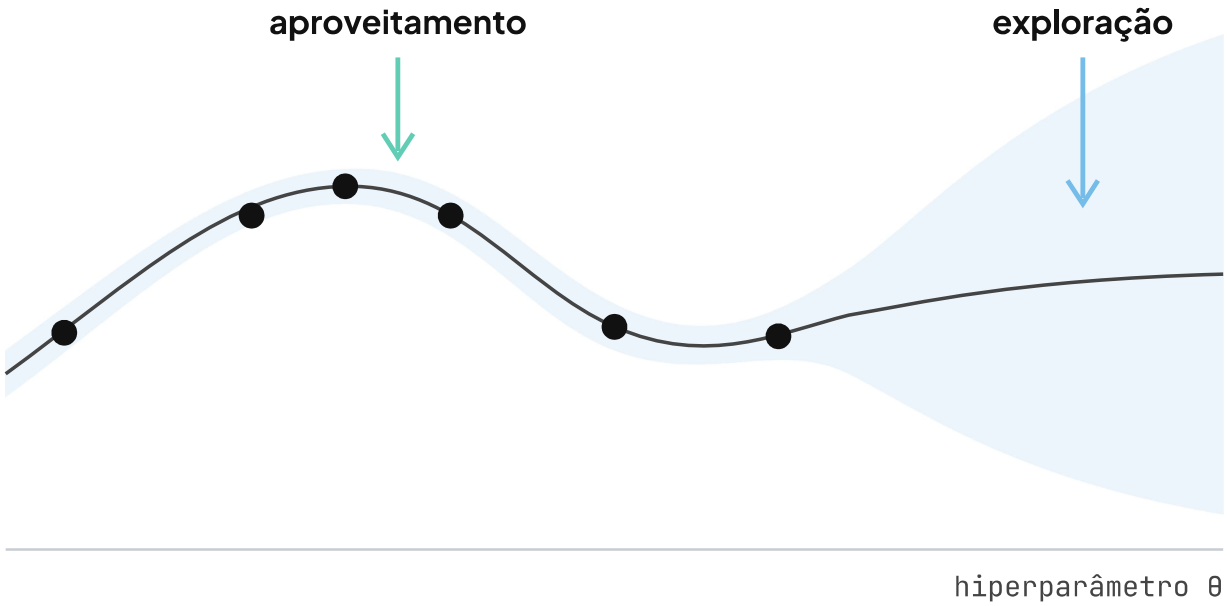
Consultar o modelo de busca é barato; avaliar J é caro.

Gastamos um pouco de cálculo para escolher melhor e economizar avaliações.

O próximo ponto depende dos resultados anteriores: não é uma busca aleatória.

BLOCO 14 · OTIMIZAÇÃO BAYESIANA

Exploração e aproveitamento



Aproveitamento

investigar perto do que já foi bom

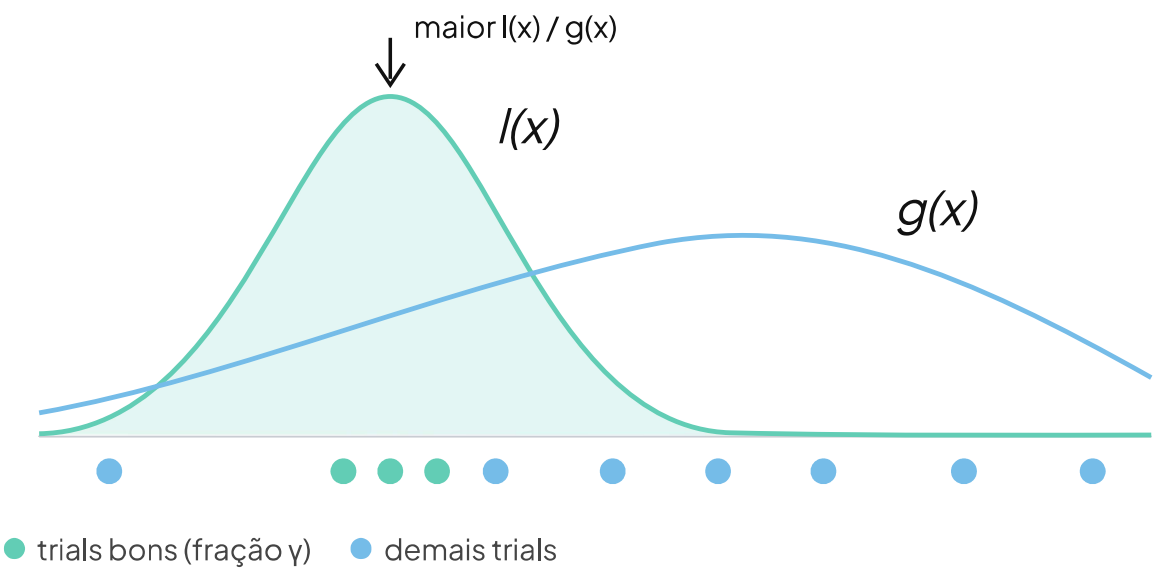
Exploração

testar onde a incerteza é alta

- avaliações feitas
- estimativa do modelo
- incerteza

BLOCO 15 · TPE

TPE: Tree-structured Parzen Estimator



$$x^* = \arg\max\, l(x) \,/\, g(x)$$

cand.	l(x)	g(x)	l/g
A	2,4	0,8	**3,00**
B	1,0	1,2	0,83
C	0,6	0,3	2,00

Valores ilustrativos. A é comum entre os bons e raro entre os demais.

BLOCO 15 · TPE

TPE não é um Processo Gaussiano

	TPE	Processo Gaussiano
O que modela	$p(x \mid y)$: distribuições dos bons e dos demais	$p(y \mid x)$: a própria função objetivo
Como escolhe	maior razão l(x) / g(x)	critério como Expected Improvement
Espaços	categóricos, inteiros, condicionais	contínuos, de baixa dimensão
No Optuna	`TPESampler (padrão)`	`GPSampler`

Os dois são métodos sequenciais baseados em modelo, frequentemente agrupados como otimização bayesiana.

BLOCO 16 · OPTUNA

Os componentes do Optuna

Study

direção · sampler · pruner · histórico de trials

Sampler
propõe os valores dos hiperparâmetros

→

Trial → objective(trial)
suggest define o espaço; treina e devolve um valor

→

Resultado
entra no histórico e orienta o sampler

Pruner
observa valores intermediários; pode interromper o trial

BLOCO 17 · OPTUNA

Study organiza; Trial é uma tentativa

```python
import optuna

study = optuna.create_study(
    direction="maximize"
)

study.trials_dataframe()
```

`direction`: `"maximize"` para ROC AUC, `"minimize"` para Log Loss.

nº	params	value	state
0	depth=2, lr=0.041	0.912	COMPLETE
1	depth=1, lr=0.022	0.874	COMPLETE
2	depth=3, lr=0.088	0.931	COMPLETE
3	depth=1, lr=0.030	—	PRUNED

Valores ilustrativos, apenas para mostrar a estrutura do histórico.

BLOCO 18 · OPTUNA

A função objetivo

```python
def objective(trial):
    max_depth = trial.suggest_int("max_depth", 1, 5)
    learning_rate = trial.suggest_float(
        "learning_rate", 0.01, 0.2, log=True)
    min_samples_leaf = trial.suggest_int(
        "min_samples_leaf", 1, 10)

    modelo = GradientBoostingClassifier(
        max_depth=max_depth,
        learning_rate=learning_rate,
        min_samples_leaf=min_samples_leaf,
        random_state=42)

    score = cross_val_score(
        modelo, X_train, y_train,
        cv=3, scoring="roc_auc").mean()
    return score
```

1. Pede ao trial os valores: define o espaço e recebe a escolha do sampler
2. Constrói o modelo com esses valores
3. Avalia por validação cruzada no treino
4. Devolve um número: o valor do trial

Todo hiperparâmetro sugerido deve ser usado pelo modelo: por isso aqui o modelo é de boosting.

BLOCO 18 · OPTUNA

Como o trial sugere valores

`trial.suggest_int("max_depth", 1, 5)`	inteiro de 1 a 5, **incluindo** os dois extremos
`trial.suggest_float("lr", 0.01, 0.2)`	real no intervalo, escala linear
`trial.suggest_float(…, log=True)`	escala logarítmica; exige limite inferior positivo
`trial.suggest_float(…, step=0.05)`	valores discretizados; não combina com log
`trial.suggest_categorical(…, [...])`	um elemento da lista

O nome passado é a chave que aparece em `best_params` e nos gráficos.

BLOCO 19 · OPTUNA

Executar um estudo com TPE

```
sampler = optuna.samplers.TPESampler(
    seed=42
)

study = optuna.create_study(
    direction="maximize",
    sampler=sampler
)

study.optimize(objective, n_trials=12)

study.best_params   # dicionário
study.best_value    # melhor valor
study.best_trial    # trial completo
```

`seed=42`

sorteios reprodutíveis em execução serial

`n_startup_trials`

primeiros trials aleatórios; padrão 10

`n_trials=12`

orçamento máximo de tentativas

`optimize()`

cria trial → chama objective → registra → repete

BLOCO 20 · OPTUNA

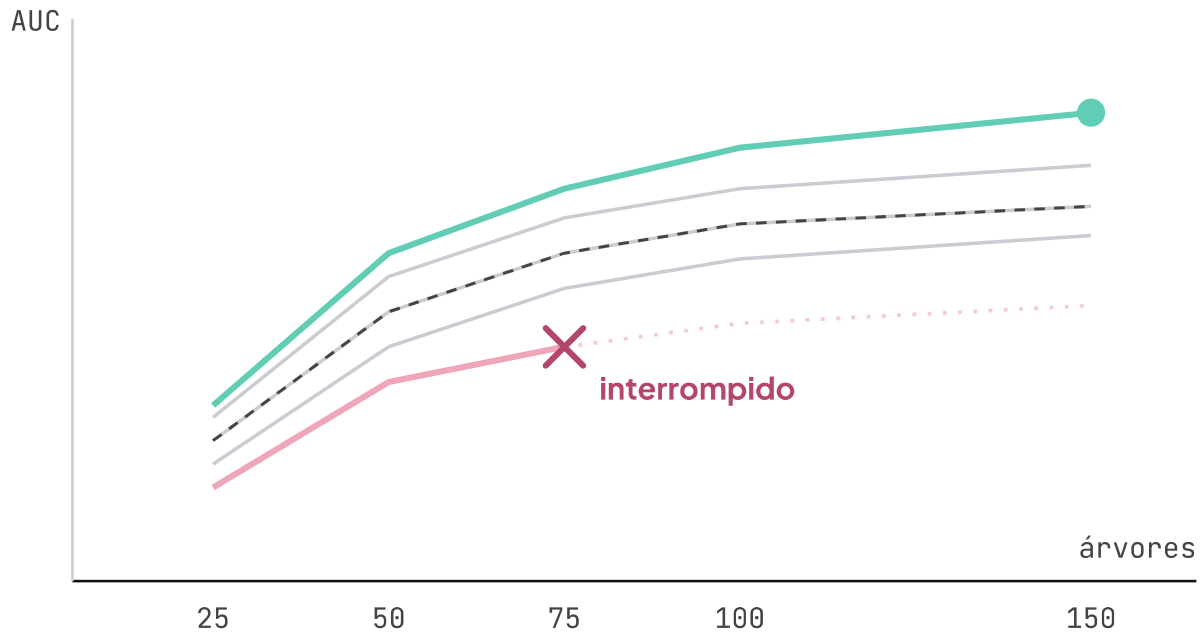
Visualizações nativas do Optuna

`plot_optimization_history`	valor de cada trial e melhor valor acumulado
`plot_param_importances`	quanto cada hiperparâmetro explica da variação do resultado
`plot_slice`	distribuição dos valores testados e relação com o resultado
`plot_parallel_coordinate`	cada linha é um trial atravessando os hiperparâmetros
`plot_intermediate_values`	curvas reportadas por trial, incluindo os interrompidos

Disponíveis em `optuna.visualization` (Plotly) e `optuna.visualization.matplotlib`.

BLOCO 21 · PRUNING

Pruning: interromper trials pouco promissores



trials anteriores

mediana por etapa

abaixo da mediana

promissor, continua

O sampler escolhe os hiperparâmetros.
O pruner só decide se o trial continua.

Curvas ilustrativas.

BLOCO 22 · PRUNING

MedianPruner: comparar com a mediana

```
pruner = optuna.pruners.MedianPruner(
    n_startup_trials=5,
    n_warmup_steps=50,
    interval_steps=25,
    n_min_trials=3
)
```

`n_startup_trials=5`
não interrompe até 5 trials terminarem

`n_warmup_steps=50`
não interrompe antes da etapa 50 do trial

`interval_steps=25`
verifica a cada 25 etapas após o aquecimento

`n_min_trials=3`
exige 3 valores reportados na etapa

Regra: interrompe se o valor intermediário for pior que a mediana dos trials completos anteriores **na mesma etapa**.

BLOCO 22 · PRUNING

Um exemplo numérico na etapa 75

Trials anteriores, ordenados, na etapa 75

0,903	0,912	**0,921**	0,935	0,940

mediana = 3º valor de 5

`0,897 < 0,921` **Trial A** interrompido

`0,930 > 0,921` **Trial B** continua

Valores ilustrativos.

BLOCO 23 · PRUNING

report() e should_prune()

```python
for etapa in etapas:
    # ... treinar até esta etapa ...
    valor = avaliar_na_validacao()

    trial.report(valor, step=etapa)

    if trial.should_prune():
        raise optuna.TrialPruned()
```

`report()`	registra o valor intermediário
`step`	inteiro com o mesmo significado em todos os trials
`should_prune()`	pergunta ao pruner
`TrialPruned`	encerra o trial como PRUNED

CONDIÇÕES

O treino precisa ter etapas intermediárias avaliáveis.

Os valores precisam ser comparáveis: mesma métrica, mesmos folds, mesmo significado de step.

Nenhum estimador é podado automaticamente: nós estruturamos o treino em etapas.

BLOCO 24 · PRUNING

Sampler e pruner têm funções diferentes

ANTES DO TRIAL

Sampler

Escolhe quais hiperparâmetros testar.

`TPESampler`

DURANTE O TRIAL

`25 → 50 → 75 → 100 → 150`

treino em etapas, com `report()` a cada checkpoint

A CADA ETAPA

Pruner

Decide se o trial deve ser interrompido.

`MedianPruner`

São independentes: TPE sem pruner, ou sampler aleatório com MedianPruner, também funcionam.

BLOCO 25 · TREINAMENTO INCREMENTAL

warm_start: acrescentar árvores sem retreinar



25	50	75	100	150
fit inicial	+25 árvores	+25 árvores	+25 árvores	+50 árvores

```
modelo = GradientBoostingClassifier(
    n_estimators=25, warm_start=True)
for checkpoint in [25, 50, 75, 100, 150]:
    modelo.set_params(n_estimators=checkpoint)
    modelo.fit(X_fold_train, y_fold_train)
```

Chegar a 150 custa 150 árvores, não 25 + 50 + 75 + 100 + 150.

Não é o early stopping do modelo (`n_iter_no_change`): aqui quem decide parar é o pruner.

BLOCO 26 · COMPARAÇÃO

Grid, Random e Optuna lado a lado

	Grid Search	Random Search	Optuna + TPE
Como escolhe	todas as combinações da grade	sorteio a partir de distribuições	propostas guiadas pelo histórico
Valores intermediários	não	sim	sim
Custo	∏ tamanhos × folds	n_iter × folds	até n_trials × folds; menos com pruning
Paralelização	trivial	trivial	possível, mas altera a dinâmica
Reprodutibilidade	folds fixos	random_state	seed e execução serial
Limitação	explosão combinatória	não aprende com os resultados	pouca informação com poucos trials

Nenhuma estratégia é sempre superior. O resultado depende do espaço, do orçamento e do problema.

BLOCO 27 · PROTOCOLO

Mesmo orçamento, três estratégias

FIXADO PARA AS TRÊS

Mesmo dataset

Mesmo espaço geral de busca

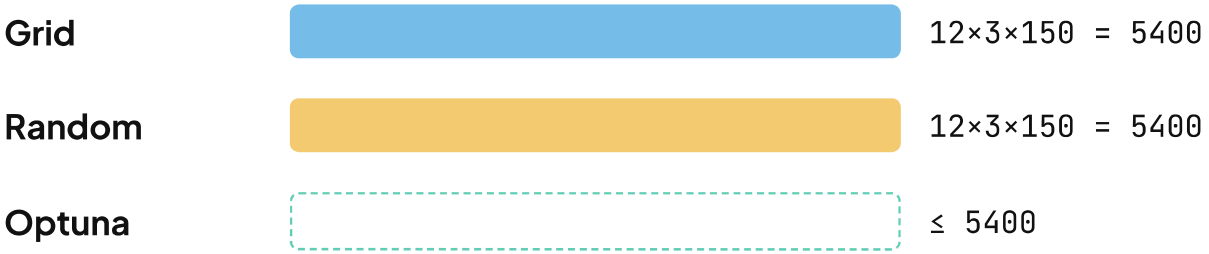
Mesmos folds

Mesma métrica: ROC AUC

Mesmo modelo: GradientBoosting

Máximo de 150 estimadores

Até 12 configurações



árvores-base treinadas durante a busca

Igualar o número de configurações não iguala o custo. Mediremos separadamente: configurações, árvores efetivamente treinadas e tempo real.

EXERCÍCIO GUIADO · 30 MIN

Mesmo orçamento, três estratégias

Um notebook vazio. Nenhum import, dataset ou modelo pronto. Mesmo modelo nas três buscas: `GradientBoostingClassifier`.

Dataset → Split → Grid Search → Random Search → Optuna → Comparação

EXERCÍCIO · ETAPA 1

Etapa 1 · Imports

```
import time
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import optuna
from optuna.trial import TrialState
from scipy.stats import randint, loguniform
from sklearn.datasets import make_classification
from sklearn.model_selection import (
    train_test_split, StratifiedKFold,
    GridSearchCV, RandomizedSearchCV, cross_val_score)
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.metrics import (
    accuracy_score, f1_score, roc_auc_score,
    log_loss, confusion_matrix)
```

`time`	medir a duração das buscas
`numpy · pandas`	contas e tabelas
`matplotlib`	gráficos
`optuna`	estudo, trials, sampler, pruner
`scipy.stats`	distribuições do Random Search
`model_selection`	divisões e buscas
`ensemble`	o modelo das três estratégias
`metrics`	avaliação final

EXERCÍCIO · ETAPA 2

Etapa 2 · Criar o dataset

```
X, y = make_classification(
    n_samples=1800,
    n_features=16,
    n_informative=8,
    n_redundant=4,
    n_repeated=0,
    n_classes=2,
    weights=[0.55, 0.45],
    flip_y=0.02,
    class_sep=1.0,
    random_state=42
)
```

16 características



8 informativas | 4 redundantes | 4 ruído

`weights`	≈ 55% classe 0, 45% classe 1
`flip_y`	2% dos rótulos sorteados: ruído
`class_sep`	separação moderada
`random_state`	mesmo dataset sempre

EXERCÍCIO · ETAPAS 3 E 4

Etapas 3 e 4 · Explorar e separar o teste

```
print("Formato de X:", X.shape)
print("Formato de y:", y.shape)
print("Classes:",
      np.unique(y, return_counts=True))

df = pd.DataFrame(
    X, columns=[f"x{i}" for i in range(16)])
df["classe"] = y
df.head(); df.describe()
```

```
Formato de X: (1800, 16)
Formato de y: (1800,)
Classes: PREENCHER APÓS RUN ALL
```

```
X_train, X_test, y_train, y_test = (
    train_test_split(
        X, y, test_size=0.20,
        stratify=y, random_state=42))
```



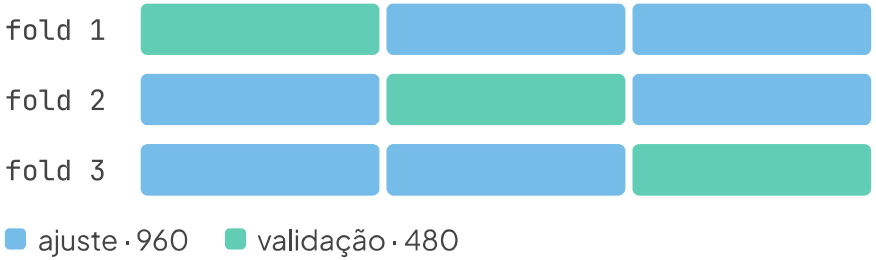
`stratify=y` mantém a proporção das classes. O teste fica reservado até a Etapa 19.

EXERCÍCIO · ETAPAS 5 E 6

Etapas 5 e 6 · Mesmos folds e uma referência

```
cv = StratifiedKFold(
    n_splits=3, shuffle=True, random_state=42)
splits = list(cv.split(X_train, y_train))

baseline = GradientBoostingClassifier(
    n_estimators=150, random_state=42)
auc_base = cross_val_score(
    baseline, X_train, y_train,
    cv=splits, scoring="roc_auc").mean()
```



A mesma lista `splits` é usada pelas três estratégias.

```
AUC baseline: PREENCHER APÓS RUN ALL
```

EXERCÍCIO · ETAPAS 7 E 8

Etapas 7 e 8 · Grid Search

```
param_grid = {
    "max_depth": [1, 2, 3],
    "learning_rate": [0.03, 0.10],
    "min_samples_leaf": [1, 5]
}
```

3 × 2 × 2 = 12 configurações

12 × 3 folds = 36 ajustes de 150 árvores; sem refit

```
best_params_: PREENCHER APÓS RUN ALL
best_score_: PREENCHER APÓS RUN ALL
tempo: PREENCHER APÓS RUN ALL
```

```
modelo_base = GradientBoostingClassifier(
    n_estimators=150, random_state=42)

grid = GridSearchCV(
    estimator=modelo_base,
    param_grid=param_grid,
    scoring="roc_auc",
    cv=splits,
    n_jobs=1,
    refit=False,
    return_train_score=True,
    error_score="raise")

inicio = time.perf_counter()
grid.fit(X_train, y_train)
tempo_grid = time.perf_counter() - inicio
```

EXERCÍCIO · ETAPAS 9 E 10

Etapas 9 e 10 · Random Search

```
param_distributions = {
    "max_depth": randint(1, 4),
    "learning_rate": loguniform(0.02, 0.15),
    "min_samples_leaf": randint(1, 6)
}
```

max_depth	inteiros 1, 2, 3
learning_rate	contínuo, escala log
min_samples_leaf	inteiros de 1 a 5

```
best_params_: PREENCHER APÓS RUN ALL
best_score_: PREENCHER APÓS RUN ALL
tempo: PREENCHER APÓS RUN ALL
```

```
random_search = RandomizedSearchCV(
    estimator=modelo_base,
    param_distributions=param_distributions,
    n_iter=12,
    scoring="roc_auc",
    cv=splits,
    random_state=42,
    n_jobs=1,
    refit=False,
    return_train_score=True,
    error_score="raise")

inicio = time.perf_counter()
random_search.fit(X_train, y_train)
tempo_random = time.perf_counter() - inicio
```

EXERCÍCIO · ETAPAS 11 E 12

Etapas 11 e 12 · Sampler e pruner

Sampler · escolhe

```
sampler = optuna.samplers.TPESampler(
    seed=42,
    n_startup_trials=5
)
```

Trials 0 a 4 aleatórios. Do trial 5 em diante, o TPE propõe valores a partir do histórico.

Pruner · interrompe

```
pruner = optuna.pruners.MedianPruner(
    n_startup_trials=5,
    n_warmup_steps=50,
    interval_steps=25,
    n_min_trials=3
)
```

Com 12 trials e 5 de aquecimento, no máximo 7 podem ser interrompidos. Quantos serão depende dos resultados.

EXERCÍCIO · ETAPA 13

Etapa 13 · Função objetivo: sugestões e modelos

```
CHECKPOINTS = [25, 50, 75, 100, 150]

def objective(trial):
    params = {
        "max_depth": trial.suggest_int("max_depth", 1, 3),
        "learning_rate": trial.suggest_float(
            "learning_rate", 0.02, 0.15, log=True),
        "min_samples_leaf": trial.suggest_int(
            "min_samples_leaf", 1, 5),
    }
    # um modelo por fold, crescendo com warm_start
    modelos = [
        GradientBoostingClassifier(
            n_estimators=CHECKPOINTS[0],
            warm_start=True, random_state=42,
            **params)
        for _ in splits
    ]
    # continua no próximo slide ...
```

Mesmos limites do Random Search. `suggest_int` inclui o limite superior: 3 e 5.

Um modelo por fold: cada um cresce sobre seus próprios dados de ajuste.

Todos começam com 25 árvores, o primeiro checkpoint.

EXERCÍCIO · ETAPA 13

Etapa 13 · Treino progressivo, report e pruning

```
    for checkpoint in CHECKPOINTS:
        aucs = []
        for modelo, (idx_tr, idx_va) in zip(modelos, splits):
            modelo.set_params(n_estimators=checkpoint)
            modelo.fit(X_train[idx_tr], y_train[idx_tr])
            proba = modelo.predict_proba(X_train[idx_va])[:, 1]
            aucs.append(roc_auc_score(y_train[idx_va], proba))
        media_auc = float(np.mean(aucs))  # média dos 3 folds

        trial.set_user_attr("ultimo_checkpoint", checkpoint)
        trial.report(media_auc, step=checkpoint)

        if checkpoint < CHECKPOINTS[-1] and trial.should_prune():
            raise optuna.TrialPruned()

    return media_auc
```

Os três folds chegam ao mesmo checkpoint antes da média.

`user_attr` guarda onde o trial parou.

Não verificamos no checkpoint 150: o treino já terminou e interromper não economizaria nada.

EXERCÍCIO · ETAPAS 14 E 15

Etapas 14 e 15 · Criar e executar o estudo

```
study = optuna.create_study(
    direction="maximize",
    sampler=sampler,
    pruner=pruner)

inicio = time.perf_counter()
study.optimize(objective, n_trials=12,
               n_jobs=1, show_progress_bar=True)
tempo_optuna = time.perf_counter() - inicio

completos = study.get_trials(states=[TrialState.COMPLETE])
podados = study.get_trials(states=[TrialState.PRUNED])
study.best_params, study.best_value
```

```
best_params: PREENCHER
best_value: PREENCHER
completos: PREENCHER
interrompidos: PREENCHER
tempo: PREENCHER
```

PREENCHER APÓS RUN ALL

Um trial para antes de 150 árvores quando sua média fica abaixo da mediana dos trials completos na mesma etapa.

EXERCÍCIO · ETAPA 16

Etapa 16 · Orçamento, trabalho e tempo

```
N_FOLDS = len(splits)

def arvores_optuna(study):
    return sum(
        t.user_attrs.get("ultimo_checkpoint", 0)
        * N_FOLDS
        for t in study.trials)

arvores_grid = 12 * N_FOLDS * 150    # 5400
arvores_random = 12 * N_FOLDS * 150  # 5400
arvores_opt = arvores_optuna(study)
```

	Grid	Random	Optuna
Configurações	12	12	12
Interrompidos	—	—	PREENCHER
Árvores treinadas	5400	5400	PREENCHER
Tempo (s)	PREENCHER	PREENCHER	PREENCHER

Árvores-base são uma aproximação do custo: profundidade e predições também pesam.

EXERCÍCIO · ETAPAS 17 A 19

Etapas 17 a 19 · Modelos finais e teste

```
melhores = {
    "Grid Search": grid.best_params_,
    "Random Search": random_search.best_params_,
    "Optuna (TPE + pruning)": study.best_params}

for nome, params in melhores.items():
    final = GradientBoostingClassifier(
        n_estimators=150, random_state=42, **params)
    final.fit(X_train, y_train)   # todo o treino
    proba = final.predict_proba(X_test)[:, 1]
    pred = (proba >= 0.5).astype(int)
    roc_auc_score(y_test, proba)
    log_loss(y_test, proba)
    accuracy_score(y_test, pred)
    f1_score(y_test, pred)
```

Probabilidade

ROC AUC: ordenação · Log Loss: confiança e calibração

Classe

Acurácia, F1 e matriz de confusão, com limiar 0,5

O teste é usado uma única vez, depois de todas as escolhas.

EXERCÍCIO · ETAPAS 20 A 22

Etapas 20 a 22 · Tabela final e conclusão

Estratégia	AUC CV	AUC teste	Acur.	F1	Log Loss	Busca (s)
Grid	—	—	—	—	—	—
Random	—	—	—	—	—	—
Optuna	—	—	—	—	—	—

PREENCHER APÓS RUN ALL com os valores da tabela gerada pelo notebook.

PARA DISCUTIR

Qual teve a melhor validação? E o melhor teste?

Qual teve o menor tempo de busca?

Quantos trials foram interrompidos e o que isso economizou?

O Grid foi competitivo?

O Optuna usou o histórico para orientar novas tentativas?

O orçamento permite conclusões robustas?

BLOCO 28 · AUTOML

Otimização de hiperparâmetros dentro do AutoML

01	02	03	04	05
Pré-processamento	**Seleção de algoritmos**	**Otimização de hiperparâmetros** esta aula	**Validação**	**Comparação de modelos**

AutoML automatiza o pipeline inteiro. Grid Search e Optuna são ferramentas para uma de suas etapas.

Continua sendo nosso: definir problema e métrica, proteger o teste e interpretar resultados.

ERROS COMUNS · 1 A 10

Erros comuns e correções (1/2)

Confundir parâmetros e hiperparâmetros
Parâmetros vêm do fit; hiperparâmetros, do construtor.

Achar que o Grid testa qualquer valor
Só os valores listados na grade.

Achar que o Random avalia todas as combinações
Avalia exatamente n_iter configurações.

Confundir sampler e pruner
Sampler escolhe valores; pruner interrompe trials.

Confundir Study e Trial
Study é o estudo inteiro; Trial é uma tentativa.

Confundir TPE com Processo Gaussiano
TPE modela p(x | y) com duas densidades; GP modela p(y | x).

Escala linear para learning_rate sem justificativa
Use log: mesma atenção a cada ordem de grandeza.

Escolher o modelo pelo teste
Escolha pela validação; o teste mede uma vez, no fim.

Validação cruzada no conjunto de teste
A validação cruzada acontece só dentro de X_train.

Comparar estratégias com splits diferentes
Use a mesma lista splits nas três.

ERROS COMUNS · 11 A 20

Erros comuns e correções (2/2)

Comparar métricas diferentes
Mesma métrica para as três estratégias.

Ignorar aleatoriedade
Fixe e documente sementes; repita se possível.

Orçamento diferente sem documentar
Mesmo número de configurações, registrado.

Trials = modelos treinados
Conte folds e árvores; trials podados treinam menos.

Pruning melhora o melhor modelo
Pruning economiza recursos; não garante qualidade.

report() sem valores coerentes
Sempre a média dos mesmos folds, no mesmo step.

Pruning em modelo sem etapas intermediárias
Estruture o treino em etapas, como com warm_start.

“Optuna sempre supera Grid”
Depende do espaço, do orçamento e do problema.

“Random sempre supera Grid”
É uma vantagem estatística, não garantia.

best_score_ = desempenho no teste
best_score_ é otimista: é o máximo de várias tentativas.

QUIZ · 5 MIN

Quiz (1/2)

01 O que é um hiperparâmetro?

02 Qual a diferença entre parâmetro e hiperparâmetro?

03 O que é espaço de busca?

04 Como funciona o Grid Search?

05 Como funciona o Random Search?

06 Por que o Random Search pode ser eficiente?

07 O que é otimização adaptativa?

08 O que significa TPE?

09 O que é um Study?

10 O que é um Trial?

QUIZ · 5 MIN

Quiz (2/2)

11 Qual a função de `objective()`?

12 O que faz `suggest_float()`?

13 Qual a diferença entre sampler e pruner?

14 O que é o MedianPruner?

15 Para que servem `report()` e `should_prune()`?

16 Por que utilizamos validação cruzada?

17 Por que não otimizar usando o teste?

18 O que significa mesmo orçamento de experimentos?

19 Por que menor tempo não significa melhor resultado?

20 Qual a relação entre otimização e AutoML?

DESAFIO FINAL · ATIVIDADE DE CASA

Desafio: otimizando um modelo no dataset Wine

178
amostras

13
características químicas

3
classes · métrica: acurácia

01 Carregar o dataset
02 Explorar os dados
03 Dividir treino e teste
04 Definir o espaço de busca
05 Executar Grid Search
06 Executar Random Search
07 Executar Optuna
08 Comparar os resultados
09 Avaliar os melhores modelos
10 Comparar os tempos
11 Explicar as diferenças
12 Concluir qual estratégia foi mais interessante

Mesmas condições para as três estratégias. O teste permanece reservado até o fim.

ENCERRAMENTO DO MÓDULO 7

O caminho desta aula

Parâmetros → Hiperparâmetros → Espaço de busca → Grid Search → Random Search → TPE → Optuna → Pruning → Comparação → AutoML

Materiais na pasta da aula: notebook do exercício guiado, desafio Wine com gabarito, roteiro oral e checklist de aprendizagem.